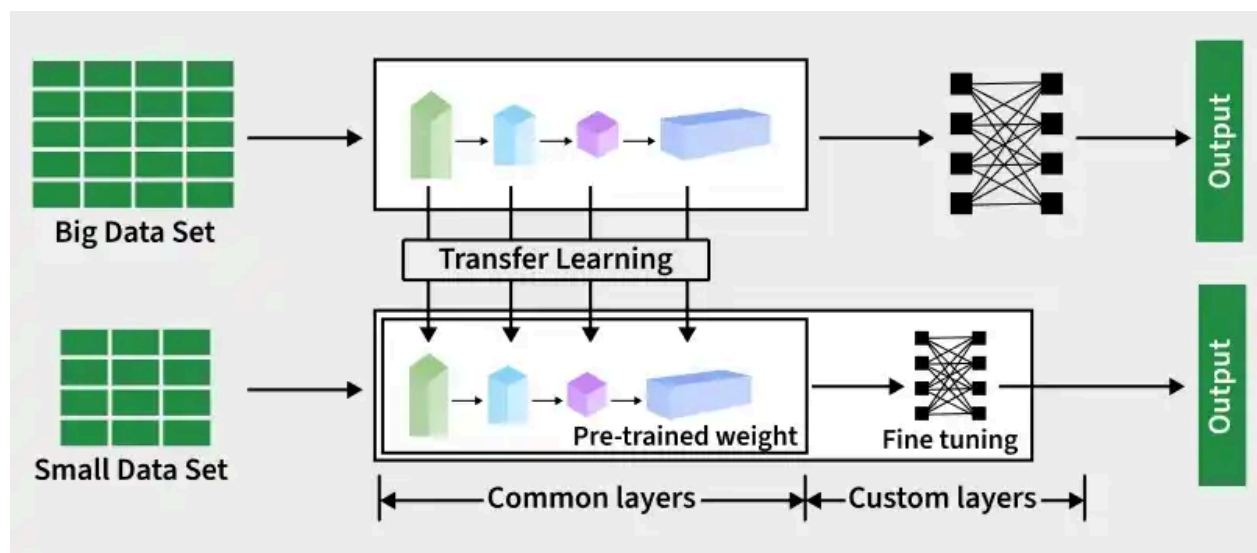


Fine-Tuning Models

Introduction

Fine-tuning is a technique that adapts a pre-trained model to a new task. It uses the knowledge learned from training on a large dataset and applies it to a smaller, task-specific dataset, improving performance while reducing training time.



Types of Fine-Tuning

1. Full Fine-Tuning

In Full Fine-Tuning, all the parameters of the pre-trained model are updated using the new dataset.

- Updates every layer of the model.
- Provides maximum flexibility and performance.
- Requires significant computational resources and training time.

2. Feature Extraction

In Feature Extraction, the pre-trained model is used as a fixed feature extractor and only the final task-specific layers are trained.

- Most layers remain frozen.
- Faster and more computationally efficient.
- Suitable when the new dataset is small.

3. Partial Fine-Tuning

In Partial Fine-Tuning, only selected layers of the model are updated while the remaining layers stay frozen.

- Balances performance and computational cost.

- Preserves general knowledge learned during pre-training.
- Commonly used in practical applications.

4. Parameter-Efficient Fine-Tuning (PEFT)

Parameter-Efficient Fine-Tuning updates only a small subset of model parameters instead of the entire model.

- Reduces memory and storage requirements.
- Faster than full fine-tuning.
- Widely used for large language models.

5. Low-Rank Adaptation (LoRA)

LoRA is a popular PEFT technique that adds small trainable matrices to the model while keeping the original weights frozen.

- Requires fewer trainable parameters.
- Reduces computational cost.
- Commonly used for fine-tuning large language models such as LLMs.

6. Prompt Tuning

Prompt Tuning learns a set of trainable prompts while keeping the model parameters unchanged.

- Requires minimal training resources.

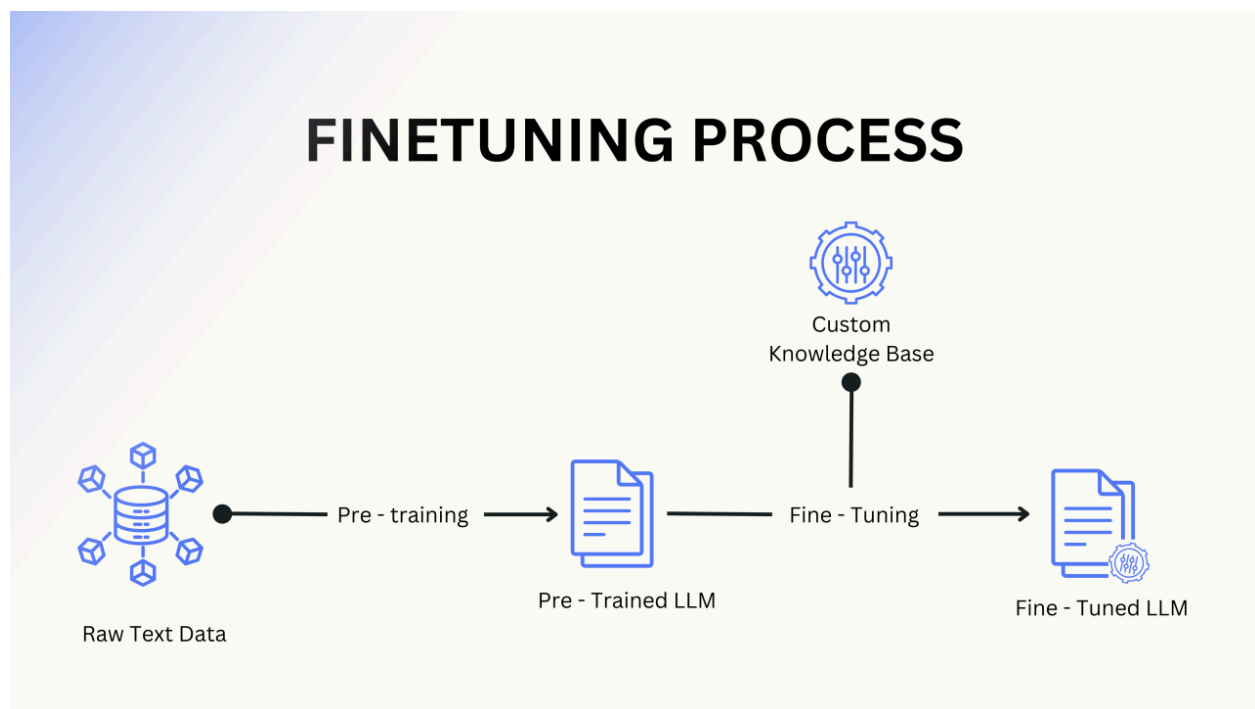
- Useful for adapting large models to new tasks.
- Maintains the original model weights.

Process to fine tune a model

1. **Setting up the environment:** Fine-tuning a model is typically an iterative process, so most open source models will be trained more than once — which means having the training data on the same ML platform will become crucial for both performance and cost. Fine-tuning a GenAI model on enterprise data requires access to proprietary information and, as your business advances on the AI maturity curve, the number of models running will only grow, increasing the demand for data access. The model training environment must be able to track the movement of data (lineage), and it must be able to handle all the model parameters in memory, so a parallel architecture is usually needed for compute efficiency.
2. **Select a base model:** Today there are many open source datasets, models and prompt libraries for different tasks: architecture, size, layers of training data and performance on relevant tasks to select a model that closely matches the characteristics of the target task.
3. **Data preparation:** Transform the data to a format suited for supervised fine-tuning. Supervised fine-tuning further trains a model to generate text conditioned on a provided prompt.
4. **Adjust model parameters:** Start with an existing model and augment or fine-tune it with enterprise data. Extend these models using techniques

like retrieval augmented generation (RAG), PEFT or standard fine-tuning.

5. **Training and evaluation:** Regularly assess the model's progress during training to track its effectiveness and implement required modifications. This involves evaluating the model's performance using a distinct validation dataset throughout the training period.



When to use fine-tuning

Fine-tuning gives the model a more focused dataset such as industry-specific terminology or task-focused interactions. This helps the model generate more relevant responses for the use case, which could be anything from customizing to supplementing the model's core knowledge to extending the model to entirely new tasks and domains.

- **Task-specific adaptation:** When you have a pretrained language model, and you want to adapt it to perform a specific task, such as sentiment analysis or text generation for a particular domain using domain-specific data. Instead of training a large model from scratch, you can start with a pretrained model and fine-tune it on your specific task, leveraging its general language understanding for the new task.
- **Bias mitigation:** Fine-tuning can be used to reduce or counteract biases present in a pretrained model by providing balanced and representative training data.
- **Data security and compliance:** When working with sensitive data, you can fine-tune a model locally on your secure infrastructure to ensure that the model never leaves your controlled environment.
- **Limited data availability:** Fine-tuning is particularly beneficial when you have limited labeled data for your specific task. Instead of training a model from scratch, you can leverage a pretrained model's knowledge and adapt it to your task using a smaller dataset.

- **Continuous learning:** Fine-tuning is useful for continuous learning scenarios where the model needs to adapt to changing data and requirements over time. It allows you to periodically update the model without starting from nothing.

LLMs also can be fine-tuned to address specific industry applications, such as in healthcare where fine-tuning on proprietary medical data can result in more accurate diagnosis and treatments. Likewise, in finance applications, fine-tuned models can be taught to detect fraud by analyzing transaction data and customer behavior.

Best practices

- **Leverage pretrained models:** Pretrained models start with knowledge from vast amounts of data and a general language understanding, allowing data teams to focus on domain-specific training.
- **Start small:** When compute resources are limited, smaller models require less power and memory, making it easier and faster to experiment and iterate on them. You could start with smaller data subsets and gradually scale up to the full dataset.
- **Use high-quality datasets:** Make sure the dataset is representative of the task and domain to minimize noise and errors.
- **Experiment with data formats:** Including diverse data input types helps the model build versatility in its responses and perform across a broader range of scenarios.
- **Use hyperparameters:** Hyperparameters must be adjusted to balance learning efficiency and prevent overfitting. Experiment with different hyperparameter values to improve model accuracy.

When not to fine-tune

To avoid any potential model “over-fitting,” refrain from adding or fine-tuning tasks that are too similar to tasks in the pretrained model as it could lose its ability to generalize from the original datasets. Expanding the training datasets can increase the accuracy of the model.

